

Agent RAG Agentique — Formule 1

Système Retrieval-Augmented Generation à raisonnement, construit avec **LangGraph** · Master IIBDCC — SMA & IAD · Évaluation de fin de module

1. Objectif et domaine

L'objectif est de concevoir un système **Agentic RAG** capable de répondre à des questions simples et complexes en s'appuyant sur une base documentaire externe, tout en ajoutant des capacités de *raisonnement, planification, usage d'outils et auto-correction*. Le graphe de raisonnement est **écrit intégralement à la main avec LangGraph**, et non via la méthode prête à l'emploi `create_agent` de LangChain : ce choix est imposé par la consigne et permet de contrôler chaque étape de la boucle (analyse, récupération, évaluation, vérification).

Le domaine retenu est la **Formule 1**, riche en vocabulaire technique (DRS, ERS, turbo), en entités à comparer (pilotes, écuries, circuits) et en données chronologiques (saisons 2005–2025), ce qui fournit un terrain idéal pour distinguer questions factuelles et questions de raisonnement. Le système est **bilingue FR/EN** : les questions peuvent être posées dans l'une ou l'autre langue sur un corpus majoritairement anglophone.

2. Démarche suivie

2.1 Construction de la base documentaire

83 pages Wikipédia (≈ 3,0 M de caractères) sont téléchargées via l'API MediaWiki, nettoyées (suppression des sections non informatives, conservation des titres de section comme contexte) et classées en **9 catégories** métier : règlement, écurie, pilote, circuit, glossaire, technique, **stratégie**, saison (2005–2025) et histoire. La catégorie devient une métadonnée exploitée par les outils pour filtrer la recherche.

2.2 Prétraitement et vectorisation

Les documents sont découpés en **6 493 chunks** (800 caractères, recouvrement 120), chacun préfixé de son titre et de sa section pour rester auto-suffisant. La vectorisation utilise un modèle **multilingue local** (paraphrase-multilingual-MiniLM) exécuté via **ONNX / FastEmbed**, puis les vecteurs sont indexés dans **Chroma**.

2.3 Création des outils

L'agent dispose de **5 outils** :

- `rechercher_documents` — recherche hybride sur tout le corpus ;
- `rechercher_par_categorie` — recherche filtrée par catégorie ;
- `comparer_entites` — récupère en parallèle les faits sur deux entités ;
- `inventaire_corpus` — liste le contenu disponible ;
- `calculer_points_championnat` — calcul *déterministe* des points (barème officiel).

2.4 Modèle de langage

Une fabrique découple le graphe du fournisseur : le LLM est interchangeable (**Groq** Llama 3.3 70B, Google Gemini ou Ollama) via un simple fichier `.env`. Trois rôles distincts (raisonneur, évaluateur, juge) sont exposés, à température nulle pour les décisions structurées.

3. Fonctionnement du système — architecture du graphe

Le state LangGraph transporte, en plus de l'historique conversationnel (**mémoire** par `thread_id`), l'espace de travail d'un tour : question rendue autonome, langue, complexité, plan, documents récupérés et compteurs de garde-fou. Chaque nœud est une fonction pure qui ne renvoie que les champs qu'il modifie ; des *réducteurs* gèrent la fusion (dont la déduplication des chunks).

Approche Agentic RAG — au-delà d'un RAG linéaire, le graphe ajoute : (1) une *analyse* qui route simple/complexe et résout les références via la mémoire ; (2) une *planification* décomposant les questions complexes ; (3) une *évaluation* de la pertinence des documents déclenchant une *reformulation* si besoin (auto-correction de type CRAG) ; (4) une *vérification d'ancrage* anti-hallucination régénérant une réponse non soutenue par les sources. Des budgets bornent chaque boucle pour garantir la terminaison.

```
START
↓
analyze_query (langue, autonomie,
               simple / complexe)
↓
plan_research (si complexe)
↓
agent ----- generate
  ↑ (pertinent) ↓
tools → grade_documents verify_grounding
        ↓ (hors sujet) ↓
        rewrite_query   END
```

4. Choix techniques et justifications

Décision	Raison
LangGraph manuel (pas <code>create_agent</code>)	Contrôle explicite de chaque étape agentique ; insertion des nœuds d'évaluation, de reformulation et de vérification impossibles à placer dans une boucle ReAct préconstruite. Exigé par la consigne.
Embeddings ONNX / FastEmbed	PyTorch ne publie plus de <i>wheel</i> pour macOS x86_64 / Python 3.13 : le projet serait ininstallable. ONNX est aussi plus léger et plus rapide sur CPU.
Recherche hybride + RRF	La recherche dense seule échoue sur les sigles (DRS, ERS) et les noms propres ; BM25 les rattrape. La fusion par <i>Reciprocal Rank Fusion</i> combine les rangs sans normaliser des scores hétérogènes.
Normalisation des vecteurs	FastEmbed n'applique pas de normalisation finale : la similarité était dominée par la norme, et les textes génériques ressortaient devant les passages spécialisés. La normalisation restaure un vrai cosinus.
Plafond de chunks / document	Le corpus est déséquilibré (la page « Formule 1 » pèse ≈ 390 chunks) ; sans plafond un document généraliste monopolise le top-k au détriment des pages spécialisées.
Boost par millésime	Avec 22 saisons quasi identiques, une question « champion 2010 » pouvait ramener une autre année ; un bonus déterministe réancrer la recherche sur le bon millésime sans dépendre du LLM.
Sorties structurées (Pydantic)	Les décisions binaires (pertinence, ancrage) et l'analyse de question sont contraintes par un schéma, ce qui rend le routage du graphe déterministe et robuste.
Calcul de points déporté en Python	Un LLM se trompe régulièrement sur une somme de dix termes ; l'outil applique le barème officiel de façon exacte.

5. Résultats de l'évaluation

Le système est évalué sur **10 questions simples et 10 questions complexes** (bilingues). Trois dimensions sont mesurées : la **latence** bout-en-bout, la **pertinence des documents** récupérés (`hit@k` , `MRR`) et la **qualité de la réponse** notée sur 5 par un LLM juge (fidélité aux sources, complétude, clarté).

Modèle : `groq:llama-3.3-70b-versatile` (roles legers: `llama-3.1-8b-instant`) · 2 questions évaluées.

Catégorie	Latence (s)	hit@k	MRR	Fidélité /5	Complétude /5	Clarté /5
Questions simples (2)	78.04	0.75	0.58	4.00	4.00	5.00
Global (2)	78.04	0.75	0.58	4.00	4.00	5.00

Lecture. `hit@k` = fraction des documents attendus effectivement récupérés ; `MRR` = qualité du classement (1,0 = source pertinente en tête). Les questions complexes présentent une latence supérieure car elles déclenchent la planification et plusieurs tours d'outils.

6. Limites et pistes d'amélioration

Limites

- **Modèle d'embeddings symétrique** : la recherche dense favorise encore parfois les passages génériques ; le filtrage par catégorie et BM25 compensent mais ne suppriment pas le biais.
- **Appels d'outils du LLM parfois mal formés** (quirk Llama/Groq) : gérés par une capture d'erreur qui bascule vers la rédaction, mais au prix d'une réponse moins complète.
- **Latence** : la boucle agentique multiplie les appels LLM (≈ 30 s / question complexe).
- **Corpus figé** : pas de mise à jour en temps réel (résultats de courses à venir absents).

Pistes d'amélioration

- Passer à un modèle d'embeddings **asymétrique** (e5, BGE) pour un meilleur rappel question→passage.
- Ajouter un **re-ranker** (cross-encoder) après la fusion pour affiner le top-k.
- **Streaming** de la réponse finale token par token pour réduire la latence perçue.
- Étendre la mémoire à une **persistance disque** (SQLite) pour des sessions durables.
- Brancher des **sources temps réel** (API de résultats) pour les questions d'actualité.